

MODULE 1

Python Automation with Selenium

Name	Debasmita Pal
Program / Year	B.Tech CSE (IoT, CS & BT)/ 4th Year
Institution	Institute of Engineering & Management, Kolkata
Enrollment No	12023002029035
Module Covered	Module 1 – Automation with Selenium (Experiments 1.1 – 1.9)
Date	15th September 2026

Index — Module 1 Experiments

Exp. No.	Experiment Title	Status
1.1	Introduction to Selenium and Its Different Flavors	Completed
1.2	Selenium WebDriver Setup and Cross-Browser Launch	Completed
1.3	Locators and Object Identification – The Multi-Locator Login	Completed
1.4	Handling Web Page Controls – Checkboxes and Dropdowns	Completed
1.5	Handling Advanced Controls – Alerts, Windows/Tabs, iFrames, Drag-and-Drop	Completed
1.6	Handling Dynamic Elements – Web Table Traversal	Completed
1.7	Synchronization – Implicit vs Explicit Waits	Completed
1.8	Taking Screenshots and Exception Handling	Completed
1.9	Advanced Interactions – Data-Driven Input, Mouse Hover, JavaScript Execution	Completed

Experiment 1.1 : Introduction to Selenium and Its Different Flavors

1. Problem Statement

The task is to get familiar with Selenium and its main tools, and then run a simple Python program that opens a browser, visits a website and displays its page title.

2. Objective

The aim is to understand the basic differences between Selenium IDE, RC, WebDriver and Grid, and to run a first WebDriver program using Python.

3. Tools, Software & Concepts Used

- Python 3.x
- Selenium 4 (pip install selenium)
- Google Chrome browser (Selenium Manager auto-resolves the driver)
- VS Code / PyCharm IDE

- Selenium IDE – record-and-playback browser extension, good for quick/simple tests.
- Selenium RC (Remote Control) – legacy tool that used a server to inject JavaScript into the browser (now deprecated).
- Selenium WebDriver – communicates directly with the browser via native OS-level calls / the W3C WebDriver protocol; the current industry standard.
- Selenium Grid – allows distributed and parallel execution of tests across multiple machines/browsers.

4. Source Code / Implementation Steps

```
from selenium import webdriver

# Launch Chrome browser using Selenium 4 WebDriver
driver = webdriver.Chrome()
driver.maximize_window()

# Navigate to the target application
driver.get("https://www.saucedemo.com/")

# Read and print the page title
print("Page title is:", driver.title)

driver.quit()
```

5. Output

5.1 TERMINAL OUTPUT

```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\exp1.1.py
Page title is: Swag Labs
Process finished with exit code 0
```

Fig 1.1: Terminal Output — SauceDemo page title displayed as “Swag Labs”.

5.2 SELENIUM OUTPUT

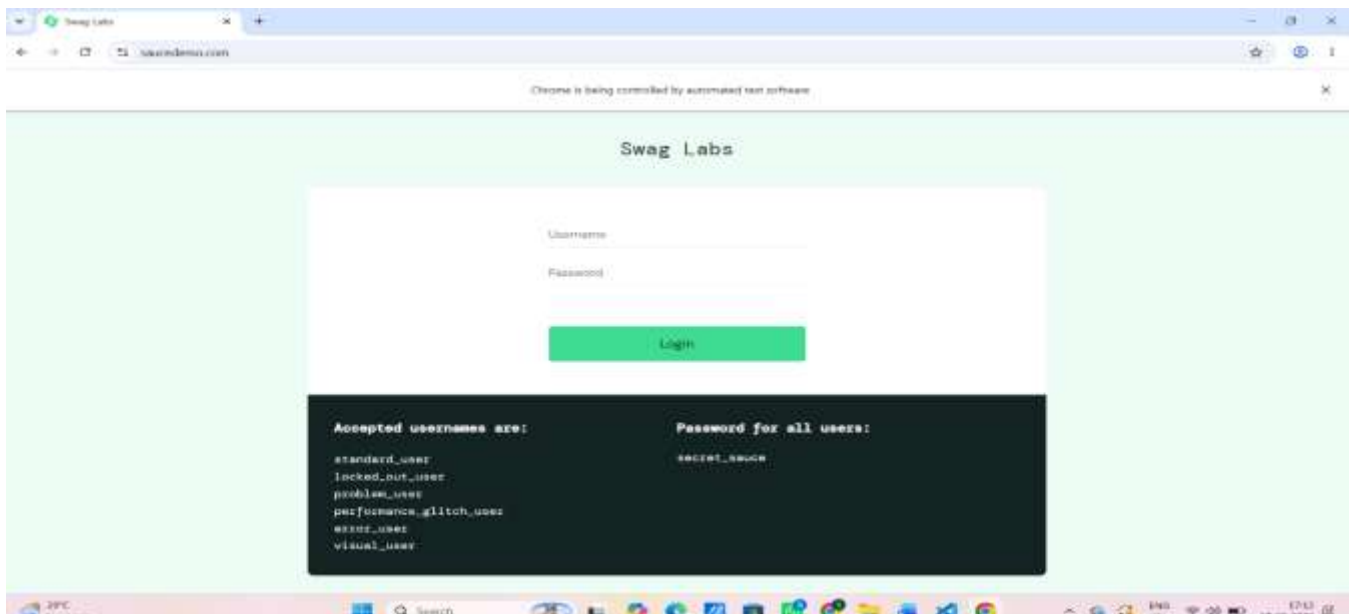


Fig 1.2: Selenium Output — Chrome browser launched via WebDriver and SauceDemo loaded.

6. Result , Observation and Conclusion

Result: The program ran successfully. Chrome opened the SauceDemo website and the terminal displayed the page title “Swag Labs”.

Observation: The Selenium browser loaded the requested website correctly, while the terminal showed the title returned by WebDriver. The driver opened the URL, read the page title, and closed the browser.

Conclusion: From both the terminal output and the Selenium output, I confirmed that the basic Selenium WebDriver program worked successfully. This experiment helped me understand how Python controls a browser through Selenium and how `driver.get()` and `driver.title` are used.

The program ran successfully. Chrome opened the SauceDemo page and the title "Swag Labs" was printed in the console. This helped me understand how WebDriver is used to control a browser directly from the Python program.

Experiment 1.2 : Selenium WebDriver Setup and Cross-Browser Launch

1. Problem Statement

Before starting automation, Selenium needs to be installed correctly and the browser should be able to start from Python. I also wanted to check that the same basic code can be used with different browsers.

2. Objective

The aim is to set up Selenium with Python and create one reusable function that can open Chrome, Edge or Firefox depending on the browser name given to it.

3. Tools, Software & Concepts Used

- Python 3.x, pip
- selenium==4.x package
- Google Chrome, Microsoft Edge, Mozilla Firefox
- Selenium Manager (auto driver-binary management, no manual PATH setup needed)
- Selenium 4 architecture: test script → language binding (Python) → JSON/W3C protocol → browser driver (chromedriver/msedgedriver/geckodriver) → actual browser.
- Selenium Manager removes the need to manually download and place driver executables in the system PATH.

4. Source Code / Implementation Steps

```
from selenium import webdriver

def get_driver(browser="chrome"):
    if browser == "chrome":
        return webdriver.Chrome()
    elif browser == "edge":
        return webdriver.Edge()
    elif browser == "firefox":
        return webdriver.Firefox()
    raise ValueError("Unsupported browser: " + browser)

for b in ["chrome", "edge"]:
    driver = get_driver(b)
    driver.get("https://www.google.com")
    print(b, "->", driver.current_url)
    driver.quit()
```

5. Output

5.1 TERMINAL OUTPUT

```

C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\exp1.1.py
chrome -> https://www.google.com/
edge -> https://www.google.com/

Process finished with exit code 0

```

Fig 1.3: Terminal output showing Chrome and Edge URLs.

5.2 SELENIUM OUTPUT

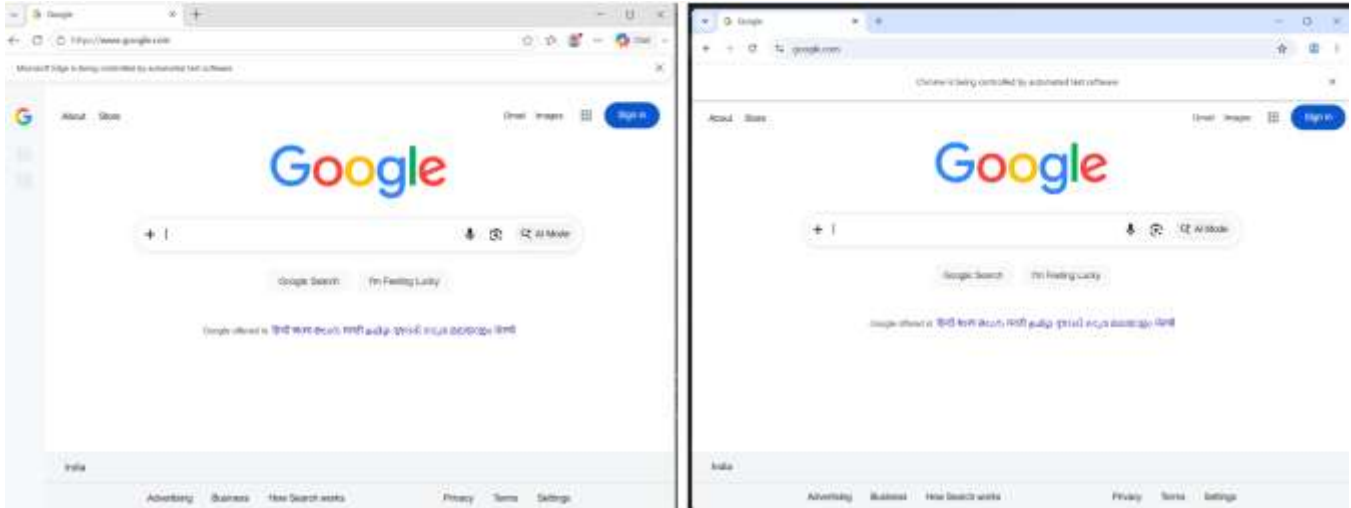


Fig 1.4: Same script launching and closing Chrome and Edge browsers in sequence.

6. Result , Observation and Conclusion

Result: Selenium WebDriver was installed successfully and the browser-launching program executed correctly.

Observation: The terminal showed the URLs reached by Chrome and Edge, while the Selenium browser output confirmed that the browsers launched through WebDriver. The `get_driver()` function selected the browser according to the value supplied to it.

Conclusion: The terminal and Selenium outputs confirmed that the WebDriver setup and cross-browser launch were successful. I learned how one Python function can select different browsers and how Selenium Manager can manage browser drivers automatically.

Selenium WebDriver was installed and verified successfully on the system. The same automation logic executed correctly on both Chrome and Edge without any code change, confirming that Selenium 4's Selenium-Manager-based architecture provides true cross-browser support.

Experiment 1.3 : Locators and Object Identification – The Multi-Locator Login

1. Problem Statement

The task is to automate a simple login page. I need to find the username, password and login button using different locator methods and then check that the login was successful.

2. Objective

The aim is to practise ID, NAME and XPATH locators and use an assertion to verify that the inventory page opens after login.

3. Tools, Software & Concepts Used

- Python 3.x, Selenium 4
- ChromeDriver (via Selenium Manager)
- SauceDemo demo login page (<https://www.saucedemo.com/>)
- By.ID, By.NAME, By.TAG_NAME, By.CLASS_NAME, By.LINK_TEXT, By.CSS_SELECTOR, By.XPATH locator strategies.
- Absolute vs relative XPath; CSS selector wildcards and child-node selection.

4. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://www.saucedemo.com/")

driver.find_element(By.ID, "user-name").send_keys("standard_user")
driver.find_element(By.NAME, "password").send_keys("secret_sauce")
driver.find_element(By.XPATH, "//*[@id='login-button']").click()

assert "inventory.html" in driver.current_url
print("Login successful. Current URL:", driver.current_url)

driver.quit()
```

5. Output

5.1 TERMINAL OUTPUT

```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\expl1.1.py
Login successful. Current URL: https://www.saucedemo.com/inventory.html

Process finished with exit code 0
```

Fig 1.5: Terminal output showing Login Successful and the current URL.

5.2 SELENIUM OUTPUT

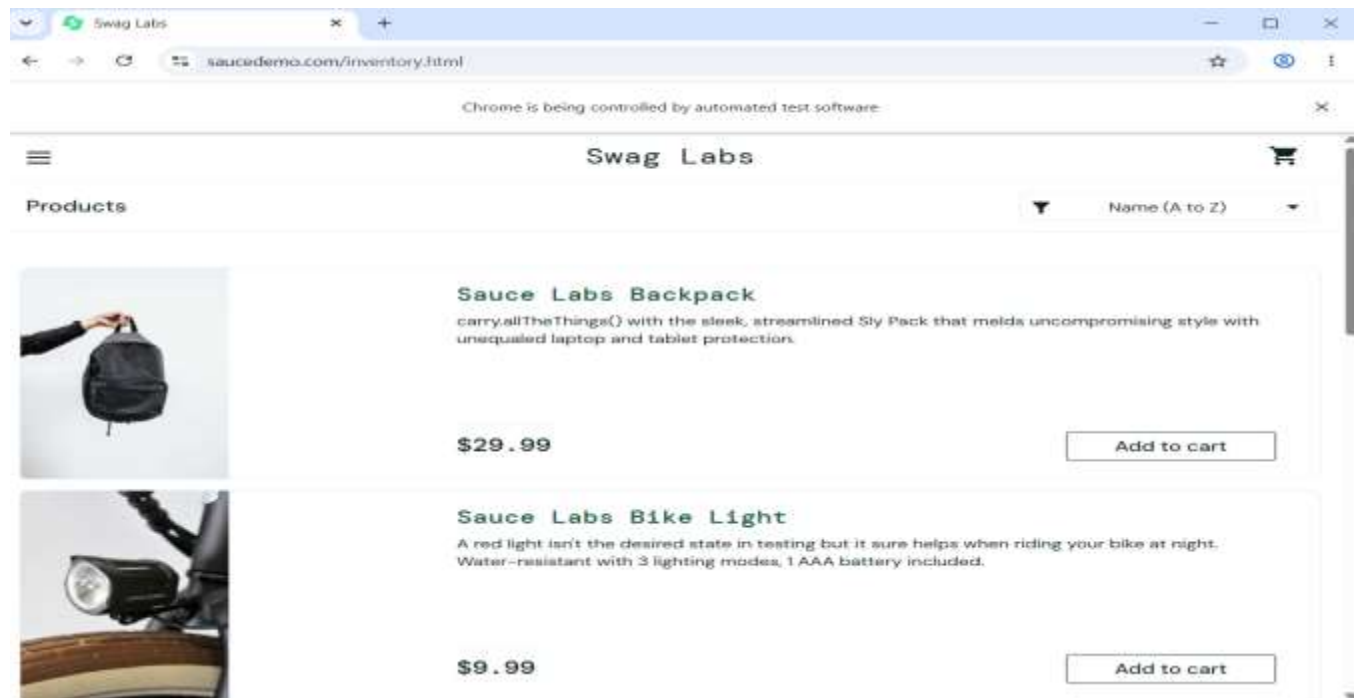


Fig 1.6: Inventory page loaded after successful login; URL assertion passed.

6. Result , Observation and Conclusion

Result: The login automation completed successfully and the inventory page was reached.

Observation: The username, password, and login button were located using different locator strategies, and the terminal confirmed that the URL assertion passed. The Selenium browser output showed the inventory page after login.

Conclusion: The terminal and Selenium outputs confirmed that the multi-locator login was successful. I learned how ID, NAME, and XPath can be used to locate webpage elements and how a URL assertion can verify that the expected page was reached.

The login worked and all three locators found the required elements. The URL check also passed. This experiment helped me see that different elements can be located in different ways depending on the HTML structure.

Experiment 1.4 :Handling Web Page Controls – Checkboxes and Dropdowns

1. Problem Statement

The task is to work with common form controls, especially checkboxes and a normal HTML drop-down, and check their state after performing the actions.

2. Objective

The aim is to select the checkboxes if they are not already selected and choose a particular option from the drop-down using Selenium's Select class.

3. Tools, Software & Concepts Used

- Python 3.x, Selenium 4
- selenium.webdriver.support.ui.Select
- the-internet.herokuapp.com demo pages (checkboxes, dropdown)
- XPath using contains() and starts-with() for elements without stable IDs.
- Parent/sibling node traversal in XPath.
- Select class for handling native HTML <select> drop-downs.

4. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import Select

driver = webdriver.Chrome()

# Checkboxes
driver.get("https://the-internet.herokuapp.com/checkboxes")
checkboxes = driver.find_elements(By.CSS_SELECTOR, "input[type='checkbox']")
for cb in checkboxes:
    if not cb.is_selected():
        cb.click()
    print("Checkbox selected:", cb.is_selected())

# Dropdown
driver.get("https://the-internet.herokuapp.com/dropdown")
dropdown = Select(driver.find_element(By.ID, "dropdown"))
dropdown.select_by_visible_text("Option 2")
print("Selected option:", dropdown.first_selected_option.text)

driver.quit()
```

5. Output

5.1 TERMINAL OUTPUT

```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\exp1.1.py
Checkbox selected: True
Checkbox selected: True
Selected option: Option 2

Process finished with exit code 0
```

Fig 1.7: Terminal output showing checkbox states and the selected dropdown option.

5.2 SELENIUM OUTPUT

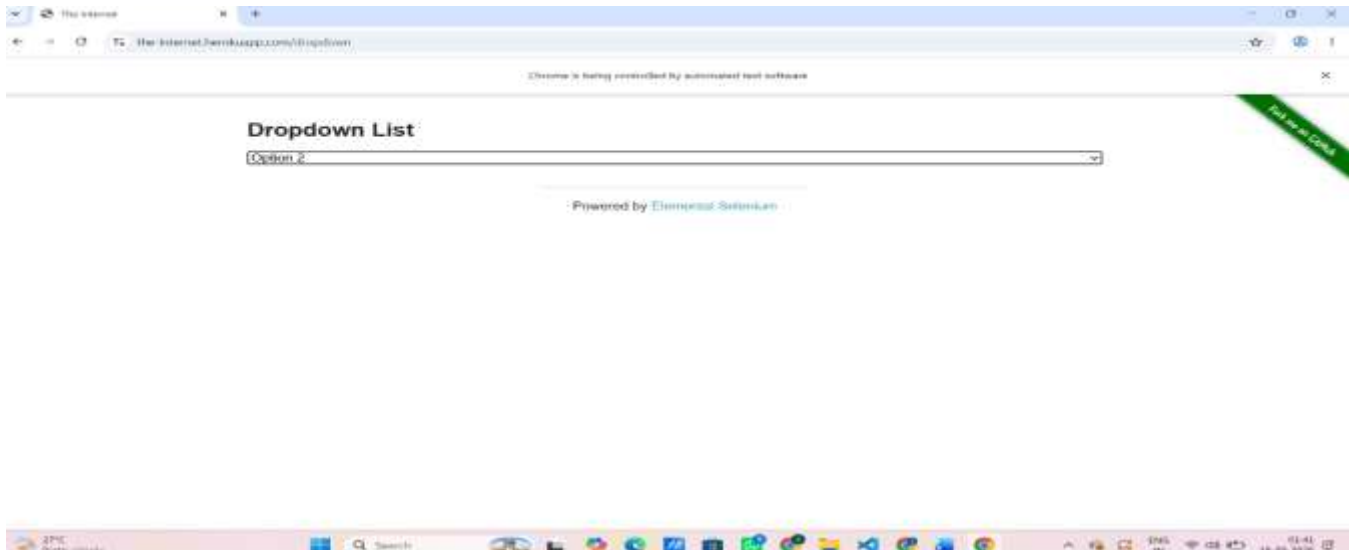


Fig 1.8: 'Option 2' selected in dropdown.

6. Result , Observation and Conclusion

Result: The checkboxes were selected successfully and “Option 2” was selected from the dropdown.

Observation: The terminal displayed the checkbox states and selected dropdown option, while the Selenium browser output showed the controls in their updated state. The script used is `is_selected()` and the `Select` class.

Conclusion: The terminal and Selenium outputs confirmed that both webpage controls were handled correctly. I learned how Selenium can check and change checkbox states and how the `Select` class can select an option from a normal HTML dropdown.

The checkboxes were selected successfully and the script confirmed their state with `is_selected()`. I also selected Option 2 from the drop-down. This showed me that a normal HTML select element can be handled conveniently with the `Select` class.

Experiment 1.5 :Handling Advanced Controls – Alerts, Windows/Tabs, iFrames and Drag-and-Drop

1. Problem Statement

Real-world applications use JavaScript alerts, multiple browser tabs/windows, embedded iframes and drag-and-drop widgets, all of which behave differently from standard page elements and require special handling.

2. Objective

The aim is to practise switching to an alert, changing between browser windows, entering text inside an iframe and performing drag-and-drop with ActionChains.

3. Tools, Software & Concepts Used

- Python 3.x, Selenium 4
- `selenium.webdriver.common.action_chains.ActionChains`
- `the-internet.herokuapp.com` (`javascript_alerts`, `windows`, `iframe`, `drag_and_drop`)
- `driver.switch_to.alert` to accept/dismiss JS alerts and confirm boxes.
- `driver.window_handles` and `driver.switch_to.window()` for multi-tab handling.
- `driver.switch_to.frame()` / `default_content()` for iframe handling.
- `ActionChains` for mouse-based interactions such as drag-and-drop and hover.

4. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains

driver = webdriver.Chrome()

# =====
# 1. JavaScript Alert
# =====

driver.get("https://the-internet.herokuapp.com/javascript_alerts")

driver.find_element(
    By.XPATH,
    "//*[@text()='Click for JS Alert']"
).click()

alert = driver.switch_to.alert

print("Alert text:", alert.text)
```

```
alert.accept()
```

```
print("JavaScript Alert handled successfully")
```

```
# =====
```

``` # 2. Multiple Windows / Tabs ```

```
# =====
```

```
driver.get("https://the-internet.herokuapp.com/windows")
```

```
main_window = driver.current_window_handle
```

```
driver.find_element(By.LINK_TEXT, "Click Here").click()
```

```
for handle in driver.window_handles:
```

```
    if handle != main_window:
```

```
        driver.switch_to.window(handle)
```

```
        print("New tab title:", driver.title)
```

```
        driver.close()
```

```
        print("New window opened and closed successfully")
```

```
driver.switch_to.window(main_window)
```

```
print("Switched back to main window successfully")
```

```
# =====
```

``` # 3. iFrame ```

```
# =====
```

```
driver.get("https://the-internet.herokuapp.com/iframe")
```

```
driver.switch_to.frame("mce_0_ifr")
```

```
print("Switched into iframe successfully")
```

```
body_el = driver.find_element(By.ID, "tinymce")
```

```
body_el.send_keys("Automated using Selenium iframe handling")
```

```
print("Text entered into iframe successfully")
```

```
driver.switch_to.default_content()
```

```
print("Switched back to main page successfully")
```

```

# =====
# 4. Drag and Drop
# =====

driver.get("https://the-internet.herokuapp.com/drag_and_drop")

source = driver.find_element(By.ID, "column-a")
target = driver.find_element(By.ID, "column-b")

ActionChains(driver).drag_and_drop(
    source,
    target
).perform()

print("Drag and Drop performed successfully")

# =====
# Close Browser
# =====

driver.quit()

print("Browser closed successfully")
print("All Selenium operations completed successfully!")

```

5. Output

5.1 TERMINAL OUTPUT



```

+ C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\expl1.1.py
+ Alert text: I am a JS Alert
+ JavaScript Alert handled successfully
+ New tab title: New Window
+ New window opened and closed successfully
+ Switched back to main window successfully
+ Switched into iframe successfully
+ Text entered into iframe successfully
+ Switched back to main page successfully
+ Drag and Drop performed successfully
+ Browser closed successfully
+ All Selenium operations completed successfully!

Process finished with exit code 0

```

Fig 1.9: Terminal output showing the alert text and new-tab title.

5.2 SELENIUM OUTPUT



Fig 1.10: JS alert accepted, new-tab title captured, iframe text updated, columns swapped after drag-and-drop.

6. Result , Observation and

Conclusion

Result: The alert, multiple-window, iframe, and drag-and-drop operations were completed successfully.

Observation: The terminal displayed the alert text and new-tab title, while the Selenium browser output showed the advanced interactions. The script used alert handling, window handles, iframe switching, and ActionChains.

Conclusion: The terminal and Selenium outputs confirmed that all four advanced-control tasks were completed. I learned that alerts, windows, and iframes require special WebDriver handling and that ActionChains can perform mouse-based operations such as drag-and-drop.

All four parts of the experiment were completed. I used `switch_to` for the alert, new window and iframe, and ActionChains for the drag-and-drop operation. This helped me understand why these controls need a little more handling than ordinary page elements.

Experiment 1.6 :Handling Dynamic Elements – Web Table Traversal

1. Problem Statement

The task is to read an HTML table and find a particular row by checking the value in its first column. After finding the row, the script should print values from other columns.

2. Objective

The aim is to practise `find_elements()` with table rows and columns and retrieve the email and due amount for the row containing the name Smith.

3. Tools, Software & Concepts Used

- Python 3.x, Selenium 4
- the-internet.herokuapp.com/tables demo page
- `find_elements()` returning a list of `WebElements` for row/column traversal.
- Reading cell text with `.text` and matching against a target string.

4. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
driver.get("https://the-internet.herokuapp.com/tables")

rows = driver.find_elements(By.CSS_SELECTOR, "#table1 tbody tr")
for row in rows:
    cols = row.find_elements(By.TAG_NAME, "td")
    last_name = cols[0].text
    if last_name == "Smith":
        print("Row found -> Email:", cols[3].text, "| Due:", cols[4].text)

driver.quit()
```

5. Output

5.1 TERMINAL OUTPUT



```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\exp1.1.py
Row found -> Email: $50.00 | Due: http://www.jsmith.com

Process finished with exit code 0
```

Fig 1.11: Terminal output showing the email and due amount for the Smith row.

5.2 SELENIUM OUTPUT

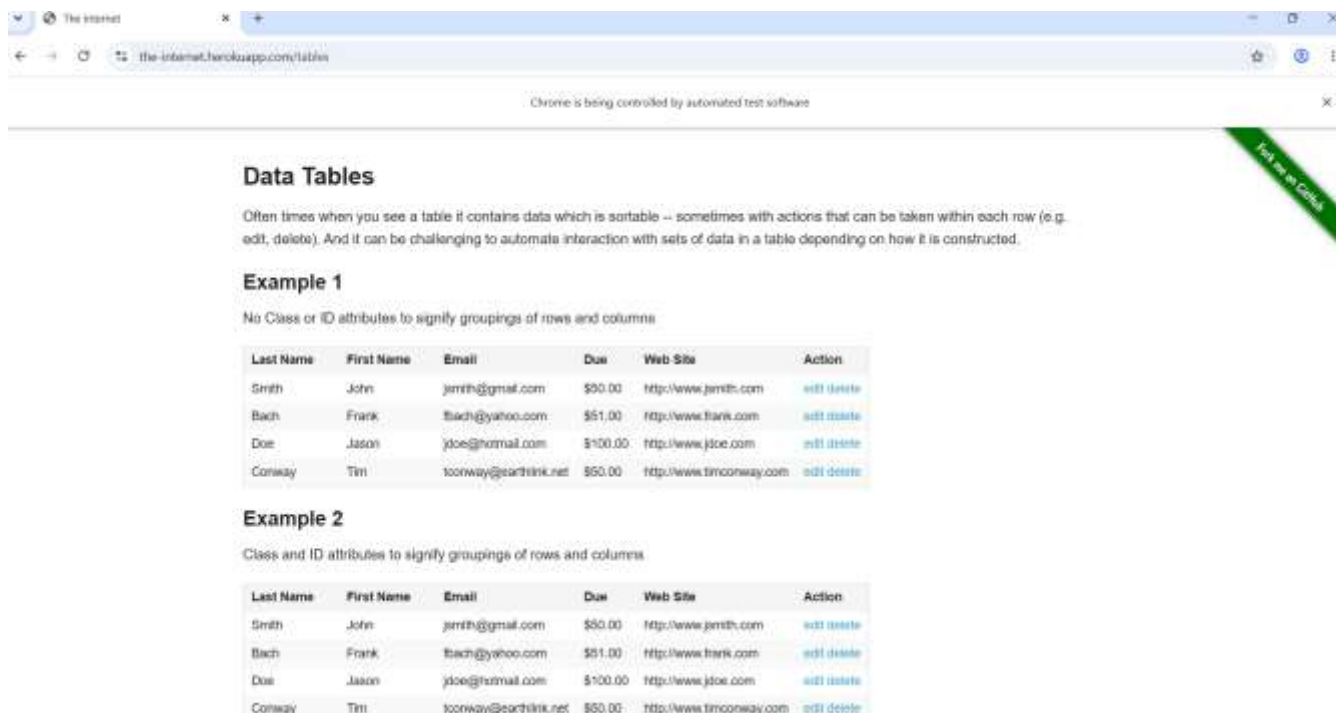


Fig 1.12: Selenium Page Providing Table Data

6. Result , Observation and Conclusion

Result: The table was successfully traversed and the row containing “Smith” was found.

Observation: The terminal displayed the email and due amount from the matching row, and the Selenium page provided the table data. The script collected rows and cells and compared the first column with the target name.

Conclusion: The terminal and Selenium outputs confirmed that the required table data was extracted successfully. I learned how `find_elements()` and loops can be used to inspect multiple rows and columns and extract information based on actual table content.

The script went through the table rows and found the row where the last name was Smith. It then printed the email and due amount from that same row. This was useful for understanding how Selenium can read table data instead of using fixed values.

Experiment 1.7 :Synchronization – Implicit vs Explicit Waits

1. Problem Statement

The page used in this experiment loads some content after a delay. If Selenium tries to access that content too early, the script may fail, so a proper wait is needed.

2. Objective

The aim is to compare implicit and explicit waits and use WebDriverWait to wait for the required element instead of using a fixed time.sleep().

3. Tools, Software & Concepts Used

- Python 3.x, Selenium 4
- selenium.webdriver.support.ui.WebDriverWait
- selenium.webdriver.support.expected_conditions
- the-internet.herokuapp.com/dynamic_loading/1
- Implicit wait – a single, global timeout applied to every find_element call for the life of the driver session.
- Explicit wait – a condition-specific wait (visibility, clickability, etc.) applied to one particular element.

4. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()
driver.implicitly_wait(5) # Implicit wait

driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")
driver.find_element(By.CSS_SELECTOR, "#start button").click()

# Explicit wait -- no time.sleep() used
wait = WebDriverWait(driver, 10)
message = wait.until(EC.visibility_of_element_located((By.ID, "finish")))
print("Message displayed:", message.text)

driver.quit()
```

5. Output

5.1 TERMINAL OUTPUT

```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\expl1.1.py
Message displayed: Hello World!

Process finished with exit code 0.
```

Fig 1.13: Terminal output showing the dynamically displayed message.

5.2 SELENIUM OUTPUT

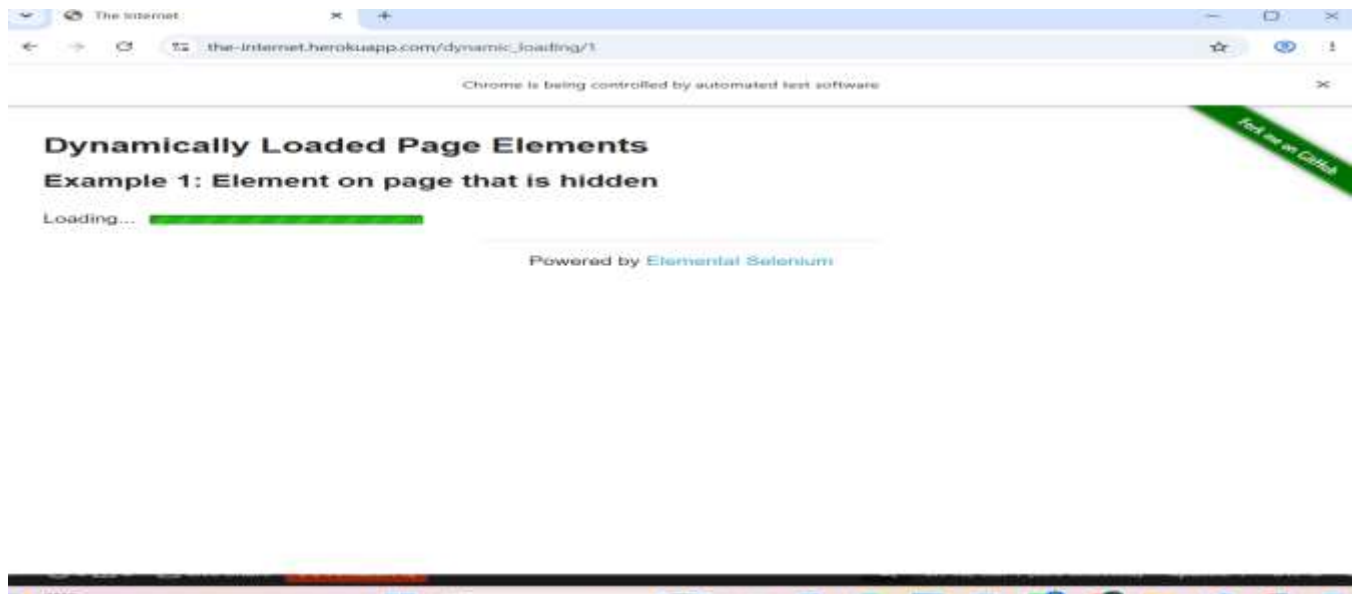


Fig 1.14: the Selenium browser showing the page during the waiting process.

6. Result , Observation and Conclusion

Result: The synchronization test completed successfully and the delayed message was displayed.

Observation: The terminal printed the message after it became available, while the Selenium browser showed the page during the waiting process. An implicit wait and an explicit `WebDriverWait` were used.

Conclusion: The terminal and Selenium outputs confirmed that the dynamic element was handled successfully. I learned the difference between implicit and explicit waits and how an explicit wait can wait for a particular condition instead of using a fixed delay.

The explicit wait worked as expected and the script printed the message after it became visible. I understood that an explicit wait is useful when I need to wait for one particular condition, while an implicit wait applies to element searches throughout the driver session.

Experiment 1.8 :Taking Screenshots and Exception Handling

1. Problem Statement

The task is to save screenshots during execution and handle a case where Selenium cannot find an element, instead of allowing the program to stop suddenly.

2. Objective

The aim is to practise screenshots and exception handling using try, except and finally, while making sure the browser is closed at the end.

3. Tools, Software & Concepts Used

- Python 3.x, Selenium 4
- `driver.save_screenshot()`
- `selenium.common.exceptions.NoSuchElementException`
- Generic screenshot capture at pass/fail points for reporting/evidence.
- try-except-finally blocks for exception handling and guaranteed cleanup (`driver.quit()`).

4. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.common.exceptions import NoSuchElementException
from selenium.webdriver.common.by import By

driver = webdriver.Chrome()
try:
    driver.get("https://www.saucedemo.com/")
    driver.save_screenshot("login_page.png")

    driver.find_element(By.ID, "nonexistent-element") # will raise exception

except NoSuchElementException as e:
    print("Handled exception:", str(e).splitlines()[0])
    driver.save_screenshot("error_state.png")

finally:
    driver.quit()
    print("Driver closed, resources released.")
```

5. Output

5.1 TERMINAL OUTPUT

```
Handled exception: Message: no such element: Unable to locate element: {"method":"css selector","selector":["id","nonexistent-element"]}
Driver closed, resources released.

Process finished with exit code 0
```

Fig 1.15: Terminal output showing the handled exception and driver cleanup message.

6. Result , Observation and Conclusion

Result: The screenshot and exception-handling test completed successfully.

Observation: The terminal showed that the missing-element exception was handled and the driver was closed. The Selenium execution captured the initial page and an error-state screenshot.

Conclusion: The terminal and Selenium outputs confirmed that the exception was handled without uncontrolled program termination. I learned how `save_screenshot()` provides evidence and how `try`, `except`, and `finally` can provide controlled error handling and browser cleanup.

The first screenshot was saved and the missing-element error was caught by the `except` block. A second screenshot was then saved for the error state. The `finally` block closed the browser, so the program finished cleanly even though an exception occurred.

Experiment 1.9 : Advanced Interactions – Data-Driven Input, Mouse Hover and JavaScript Execution

1. Problem Statement

In this experiment, the login details are kept in a separate JSON file instead of being written directly in the Selenium script. The script also demonstrates mouse hover, page scrolling and saving a test result to Excel.

2. Objective

The aim is to combine data-driven testing with a few advanced Selenium actions: reading JSON data, hovering over a menu, scrolling with JavaScript and writing the result to an Excel file.

3. Tools, Software & Concepts Used

- Python 3.x, Selenium 4
- json (standard library)
- openpyxl (pip install openpyxl)
- ActionChains, driver.execute_script()
- Reading external test data (JSON/CSV/Excel/XML) to drive test execution — Data-Driven Testing.
- ActionChains.move_to_element() for mouse-hover actions.
- driver.execute_script() to run arbitrary JavaScript (e.g., scrolling) inside the browser context.
- Writing results back to an Excel sheet using openpyxl.

4. Source Code / Implementation Steps

```
import json
import time

import openpyxl
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.common.action_chains import ActionChains

# =====
# 1. Read test data from JSON
# =====

with open("testdata.json") as f:
    data = json.load(f)

print("JSON test data loaded successfully")
```

```
# =====  
# 2. Open Browser  
# =====
```

```
driver = webdriver.Chrome()  
  
driver.get("https://www.saucedemo.com/")  
  
print("SauceDemo website opened successfully")
```

```
# =====  
# 3. Login using JSON data  
# =====
```

```
driver.find_element(  
    By.ID, "user-name"  
)send_keys(data["username"])  
  
print("Username entered successfully")  
  
driver.find_element(  
    By.ID, "password"  
)send_keys(data["password"])  
  
print("Password entered successfully")  
  
driver.find_element(  
    By.ID, "login-button"  
)click()  
  
time.sleep(2)  
  
print("Login button clicked successfully")  
print("Login completed successfully")
```

```
# =====  
# 4. Mouse Hover  
# =====
```

```
menu = driver.find_element(  
    By.ID, "react-burger-menu-btn"  
)  
  
ActionChains(driver).move_to_element(menu).perform()  
  
time.sleep(2)
```

```

print("Mouse hover performed successfully")

# =====
# 5. JavaScript Scroll
# =====

driver.execute_script(
    "window.scrollTo(0, document.body.scrollHeight);"
)

time.sleep(2)

print("JavaScript scroll to bottom performed successfully")

# =====
# 6. Write Result to Excel
# =====

wb = openpyxl.Workbook()

ws = wb.active

ws.append(["Test Case", "Result"])

ws.append([
    "Login with valid JSON-driven data",
    "Pass"
])

wb.save("results.xlsx")

print("Test result written to Excel successfully")

# =====
# 7. Close Browser
# =====

driver.quit()

print("Browser closed successfully")

print("\nAll test steps completed successfully!")

```

5. Output

5.1 TERMINAL OUTPUT

```
C:\Users\debas\AppData\Local\Programs\Python\Python314\python.exe C:\Users\debas\PycharmProjects\PythonProject\expl1.1.py
JSON test data loaded successfully
SauceDemo website opened successfully
Username entered successfully
Password entered successfully
Login button clicked successfully
Login completed successfully
Mouse hover performed successfully
JavaScript scroll to bottom performed successfully
Test result written to Excel successfully
Browser closed successfully

All test steps completed successfully!

Process finished with exit code 0
```

Fig 1.17: Terminal output showing successful JSON-driven execution and the generated Excel result.



Fig 1.18 showing successful the generated Excel File.

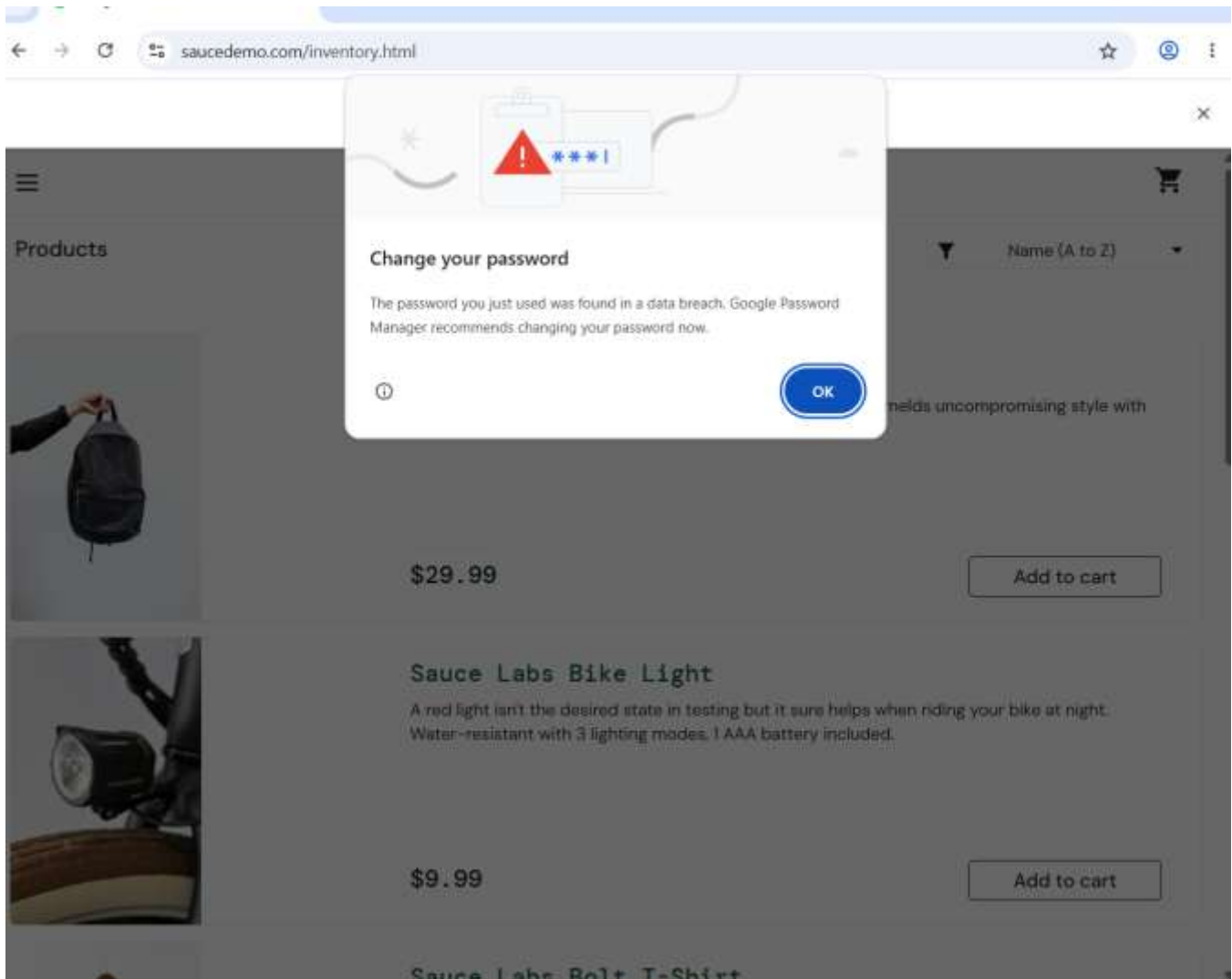


Fig 1.19: Selenium Browser Showing Login, Menu-hover & Scrolling

6. Result , Observation and Conclusion

Result: The data-driven Selenium test completed successfully and the result was saved to Excel.

Observation: The terminal showed successful JSON-driven execution, while the Selenium browser output showed the login, menu hover, and scrolling actions. The username and password were read from JSON and the result was recorded using openpyxl.

Conclusion: The terminal and Selenium outputs confirmed that the complete data-driven workflow was executed successfully. I learned how external JSON data can be used for Selenium testing and how ActionChains, JavaScript scrolling, and Excel reporting can be combined in one automation task.

The login details were taken from the JSON file and the login step completed. The script also performed the hover and scroll actions, and created an Excel file containing the test result. This experiment brought together several of the Selenium techniques covered in Module 1.